PYTHON ACTIVITY-BASED LEARNING

Complete Solutions for 10 Selected Case Studies

Student Name	Neha Jonnalagadda
Roll Number	26000923
Branch	CSM
Section	2

Selected Case Studies

1. Employee Payroll and Deduction Analysis
2. Retail Billing with Multiple Discount Rules
3. Examination Result and Grace-Mark Validation
4. Attendance Recovery Computation
5. Fuel Consumption and Route Comparison
6. Password Policy Validation Engine
7. Student Marks Matrix Analysis
8. Smart Canteen Order and Stock Management
9. Student Academic Risk Classification
10. ATM Menu-Driven Banking Simulation

Note: Rules, thresholds, and sample values are illustrative academic assumptions. Replace them if faculty instructions differ.

1. Employee Payroll and Deduction Analysis

Problem Statement

Develop a program to calculate an employee's gross salary, deductions, and net salary using basic pay and allowance/deduction rates.

Objectives

• Calculate allowances and gross salary.

• Calculate deductions using the given rates.

• Display gross salary, total deductions, and net salary.

Assumptions

• All amounts are in rupees.

• HRA is 20% and DA is 10% of basic pay.

• PF is 12% of basic pay and professional tax is ■200 when basic pay is above ■15,000; otherwise it is ■100.

Algorithm

1. Read employee name and basic pay.

2. Validate that basic pay is not negative.

3. Calculate HRA, DA, gross salary, PF, and professional tax.

4. Calculate net salary by subtracting deductions from gross salary.

5. Display the salary statement.

Flowchart

START → Read employee details → Validate basic pay → Calculate allowances and gross salary → Calculate deductions → Calculate net salary → Display salary statement → STOP

Pseudocode

```
BEGIN
  READ name, basic
  IF basic < 0: DISPLAY invalid input
  ELSE
    hra = basic * 0.20
    da = basic * 0.10
    gross = basic + hra + da
    pf = basic * 0.12
    IF basic > 15000: tax = 200
    ELSE: tax = 100
    net = gross - pf - tax
    DISPLAY gross, pf, tax, net
END
```

Python Source Code

```
name = "Neha"
basic = 20000

if basic < 0:
    print("Invalid basic pay")
else:
    hra = basic * 0.20
    da = basic * 0.10
    gross = basic + hra + da
    pf = basic * 0.12
    tax = 200 if basic > 15000 else 100
    deductions = pf + tax
    net = gross - deductions

    print("Employee:", name)
    print(f"Gross salary: Rs.{gross:.2f}")
    print(f"PF: Rs.{pf:.2f}")
    print(f"Professional tax: Rs.{tax:.2f}")
    print(f"Total deductions: Rs.{deductions:.2f}")
```

```
print(f"Net salary: Rs.{net:.2f}")
```

Test Cases

Test Input	Expected Result	Purpose
Basic pay ■20,000	Gross ■26,000; deductions ■2,600; net ■23,400	Normal salary calculation
Basic pay ■10,000	Gross ■13,000; deductions ■1,300; net ■11,700	Lower tax slab
Basic pay -■500	Invalid basic pay	Input validation

Conclusion

The program calculates salary components and net pay using conditional rules and clear arithmetic operations.

2. Retail Billing with Multiple Discount Rules

Problem Statement

Create a billing program that calculates a customer's total purchase, applies a discount based on the bill amount, and adds tax to produce the final bill.

Objectives

• Calculate the subtotal of purchased items.

• Apply the appropriate discount slab.

• Calculate tax and display the final amount.

Assumptions

• All prices are in rupees and quantities are non-negative integers.

• A 10% discount applies to subtotals of ■2,000 or more; otherwise, no discount is applied.

• GST is 5% of the discounted subtotal.

Algorithm

1. Read item prices and quantities.

2. Validate that prices and quantities are non-negative.

3. Calculate each item's amount and the subtotal.

4. Apply the discount rule.

5. Calculate GST on the discounted amount and display the bill.

Flowchart

START → Read item prices and quantities → Validate inputs → Calculate subtotal → Apply discount rule → Calculate GST → Display final bill → STOP

Pseudocode

```
BEGIN
  subtotal = 0
  FOR each item
    IF price < 0 OR quantity < 0: mark invalid
    ELSE subtotal = subtotal + price * quantity
  IF invalid: DISPLAY error
  ELSE
    IF subtotal >= 2000: discount = subtotal * 0.10
    ELSE: discount = 0
    taxable = subtotal - discount
    gst = taxable * 0.05
    total = taxable + gst
    DISPLAY subtotal, discount, gst, total
END
```

Python Source Code

```
items = [("Book", 250, 2), ("Bag", 900, 1), ("Pen", 20, 5)]
subtotal = 0
valid = True

for name, price, quantity in items:
    if price < 0 or quantity < 0:
        valid = False
        print("Invalid item:", name)
    else:
        amount = price * quantity
        subtotal += amount
        print(f"{name}: Rs.{amount:.2f}")

if valid:
    discount = subtotal * 0.10 if subtotal >= 2000 else 0
    taxable = subtotal - discount
    gst = taxable * 0.05
    total = taxable + gst
```

```
print(f"Subtotal: Rs.{subtotal:.2f}")
print(f"Discount: Rs.{discount:.2f}")
print(f"GST: Rs.{gst:.2f}")
print(f"Final bill: Rs.{total:.2f}")
```

Test Cases

Test Input	Expected Result	Purpose
Subtotal ■2,000	Discount ■200; GST ■90; total ■1,890	Discount threshold met
Subtotal ■1,000	Discount ■0; GST ■50; total ■1,050	No discount
Negative price	Invalid item	Validation

Conclusion

The billing program combines loops, arithmetic, and conditional discount rules to produce a transparent final bill.

3. Examination Result and Grace-Mark Validation

Problem Statement

Process subject marks, apply a limited grace-mark policy, and determine the student's final result without exceeding the maximum marks.

Objectives

• Validate marks and pass requirements.

• Apply grace marks only within the stated policy.

• Display original marks, final marks, and result.

Assumptions

• Each subject is out of 100 and the pass mark is 40.

• A maximum of 5 grace marks may be used in total, only when the total shortfall is at most 5.

• Invalid marks outside 0–100 are rejected.

Algorithm

1. Read the marks list.

2. Validate that every mark is between 0 and 100.

3. Find failed subjects and calculate total shortfall.

4. If shortfall is at most 5, raise those marks to 40; otherwise apply no grace.

5. Display final marks and PASS/FAIL.

Flowchart

START → Read marks → Validate range → Find failed subjects → Total shortfall ≤ 5? → Apply grace if allowed → Determine result → STOP

Pseudocode

```
BEGIN
  READ marks
  IF any mark < 0 OR mark > 100: DISPLAY invalid
  ELSE
    final_marks = copy of marks
    failed = subjects with marks < 40
    needed = sum(40 - mark for failed subjects)
    IF needed <= 5:
      set each failed subject mark to 40
      grace_used = needed
    ELSE: grace_used = 0
    IF all final marks >= 40: result = PASS
    ELSE: result = FAIL
    DISPLAY marks, final_marks, grace_used, result
END
```

Python Source Code

```
marks = [72, 38, 39, 81, 65]
pass_mark = 40
grace_limit = 5

if any(mark < 0 or mark > 100 for mark in marks):
    print("Invalid marks: enter values from 0 to 100.")
else:
    final_marks = marks.copy()
    failed = [i for i, mark in enumerate(marks) if mark < pass_mark]
    needed = sum(pass_mark - marks[i] for i in failed)

    if needed <= grace_limit:
        for i in failed:
            final_marks[i] = pass_mark
        grace_used = needed
    else:
        grace_used = 0
```

```
result = "PASS" if all(m >= pass_mark for m in final_marks) else "FAIL"
print("Original marks:", marks)
print("Final marks:", final_marks)
print("Grace marks used:", grace_used)
print("Result:", result)
```

Test Cases

Test Input	Expected Result	Purpose
[72, 38, 39, 81, 65]	Shortfall 3; final marks [72, 40, 40, 81, 65]; PASS	Grace allowed
[72, 34, 39, 81, 65]	Shortfall 7; no grace; FAIL	Grace limit exceeded
[72, 101, 39]	Invalid marks	Range validation

Conclusion

The program applies a clearly limited grace-mark policy and checks marks before calculating the final examination result.

4. Attendance Recovery Computation

Problem Statement

Calculate how many additional classes a student must attend consecutively to reach a required attendance percentage.

Objectives

• Calculate current attendance percentage.

• Estimate additional classes needed to reach the target.

• Report whether the target is already met.

Assumptions

• Classes attended and total classes are non-negative integers.

• Total classes must be greater than zero and attended classes cannot exceed total classes.

• The required attendance target is 75%; additional classes are attended without missing any.

Algorithm

1. Read attended classes and total classes.

2. Validate the values.

3. Calculate current attendance percentage.

4. If the target is already met, report zero additional classes.

5. Otherwise increase future attended and total counts together until the target is reached.

Flowchart

START → Read attended and total classes → Validate → Calculate percentage → Target met? → If not, count future attended classes until target → Display result → STOP

Pseudocode

```
BEGIN
  READ attended, total
  IF total <= 0 OR attended < 0 OR attended > total: DISPLAY invalid
  ELSE
    percent = attended / total * 100
    extra = 0
    WHILE (attended + extra) / (total + extra) * 100 < 75:
      extra = extra + 1
    DISPLAY percent, extra
END
```

Python Source Code

```
attended = 42
total = 60
target = 75

if total <= 0 or attended < 0 or attended > total:
    print("Invalid attendance data")
else:
    current = attended / total * 100
    extra = 0
    while (attended + extra) / (total + extra) * 100 < target:
        extra += 1

    print(f"Current attendance: {current:.2f}%")
    print(f"Additional classes to attend: {extra}")
    print(f"Attendance after extra classes: "
          f"{(attended + extra) / (total + extra) * 100:.2f}%")
```

Test Cases

Test Input	Expected Result	Purpose
42 attended out of 60	Current 70%; attend 12 more classes to reach 75%	Recovery calculation

Test Input	Expected Result	Purpose
48 out of 60	Current 80%; 0 additional classes	Target already met
Attended 65, total 60	Invalid attendance data	Input validation

Conclusion

The program uses a loop to calculate the minimum number of continuously attended classes required to reach the target.

5. Fuel Consumption and Route Comparison

Problem Statement

Compare routes by calculating fuel required and fuel cost, including a reserve requirement, and choose the lowest-cost feasible route.

Objectives

• Calculate fuel required for each route.

• Include a 10% fuel reserve.

• Choose the lowest-cost route that fits available fuel.

Assumptions

• Distance is in kilometres, mileage in km/litre, and fuel price in rupees/litre.

• A 10% reserve is required.

• A route is feasible when required fuel does not exceed available fuel.

Algorithm

1. Read available fuel and route records.

2. Validate distance, mileage, and fuel price.

3. Calculate base fuel and add the reserve.

4. Calculate fuel cost and check feasibility.

5. Choose the feasible route with the lowest cost.

Flowchart

START → Read route records → Validate route → Calculate fuel plus reserve → Calculate cost → Check capacity → Select lowest-cost feasible route → STOP

Pseudocode

```
BEGIN
  feasible = empty list
  FOR each route
    IF distance < 0 OR mileage <= 0 OR price < 0: skip route
    fuel = distance / mileage * 1.10
    cost = fuel * price
    IF fuel <= available_fuel: add route and cost to feasible
  IF feasible empty: DISPLAY no feasible route
  ELSE sort by cost and display cheapest route
END
```

Python Source Code

```
available_fuel = 8.0
routes = [
    ("Route A", 120, 18, 105),
    ("Route B", 135, 22, 105),
    ("Route C", 100, 14, 105)
]
feasible = []

for name, distance, mileage, price in routes:
    if distance < 0 or mileage <= 0 or price < 0:
        print(name, "has invalid data")
        continue
    fuel = distance / mileage * 1.10
    cost = fuel * price
    ok = fuel <= available_fuel
    print(f"{name}: fuel={fuel:.2f} L, cost=Rs.{cost:.2f}, feasible={ok}")
    if ok:
        feasible.append((cost, name, fuel))

if feasible:
    feasible.sort()
    cost, name, fuel = feasible[0]
```

```
    print(f"Best feasible route: {name}, {fuel:.2f} L, Rs.{cost:.2f}")
else:
    print("No route fits the available fuel.")
```

Test Cases

Test Input	Expected Result	Purpose
Route B: 135 km, 22 km/l, ■105/l; 8 L available	Fuel 6.75 L, cost ■708.75; feasible	Reserve included
Available fuel 5 L	No route fits available fuel	Capacity check
Mileage 0	Route skipped as invalid	Avoid division by zero

Conclusion

The program compares route costs fairly while accounting for reserve fuel and vehicle fuel capacity.

6. Password Policy Validation Engine

Problem Statement

Validate passwords against a defined policy that requires a minimum length, uppercase and lowercase letters, a digit, and a special character.

Objectives

• Check password length.

• Check the required character categories.

• Display all policy requirements that are not satisfied.

Assumptions

• The minimum length is 8 characters.

• A valid password must contain uppercase, lowercase, digit, and at least one special character.

• The program validates a sample password; real passwords should not be stored in plain text.

Algorithm

1. Read a sample password.

2. Check its length.

3. Use character tests to check uppercase, lowercase, digit, and special character.

4. Collect any failed requirements.

5. Display whether the password meets the policy.

Flowchart

START → Read password → Check length and character categories → Any rule failed? → Display failed rules and status → STOP

Pseudocode

```
BEGIN
  READ password
  errors = empty list
  IF length(password) < 8: add length error
  IF no uppercase: add uppercase error
  IF no lowercase: add lowercase error
  IF no digit: add digit error
  IF no special character: add special-character error
  IF errors empty: DISPLAY valid
  ELSE DISPLAY errors
END
```

Python Source Code

```
password = "Study@2026"
errors = []

if len(password) < 8:
    errors.append("At least 8 characters are required")
if not any(ch.isupper() for ch in password):
    errors.append("At least one uppercase letter is required")
if not any(ch.islower() for ch in password):
    errors.append("At least one lowercase letter is required")
if not any(ch.isdigit() for ch in password):
    errors.append("At least one digit is required")
if not any(not ch.isalnum() for ch in password):
    errors.append("At least one special character is required")

if errors:
    print("Password does not meet the policy:")
    for error in errors:
        print("-", error)
else:
    print("Password meets all policy requirements.")
```

Test Cases

Test Input	Expected Result	Purpose
Study@2026	Meets all requirements	Valid password
hello	Length, uppercase, digit, special character missing	Multiple failures
WELCOME123	Lowercase and special character missing	Category validation

Conclusion

The validator checks several password rules and explains which requirements must be met.

7. Student Marks Matrix Analysis

Problem Statement

Analyse student marks stored in a nested list, calculate each student's total and average, and identify the highest-scoring student and subject averages.

Objectives

• Calculate row-wise totals and averages.

• Find the top-performing student.

• Calculate subject-wise averages using nested lists.

Assumptions

• Each row represents one student and each column represents one subject.

• All marks are from 0 to 100 and each student has the same number of subjects.

• The sample data is illustrative.

Algorithm

1. Store student names, subject names, and marks in lists.

2. Validate that marks are within 0–100.

3. Calculate each student's total and average.

4. Find the student with the highest total.

5. Calculate the average for each subject.

Flowchart

START → Read names and marks matrix → Validate marks → Calculate student totals and averages → Find top student → Calculate subject averages → Display report → STOP

Pseudocode

```
BEGIN
  FOR each student row
    total = sum(row)
    average = total / number_of_subjects
    DISPLAY student, total, average
  top_index = index of maximum row sum
  FOR each subject column
    subject_average = sum(column) / number_of_students
    DISPLAY subject_average
  DISPLAY top student
END
```

Python Source Code

```
students = ["Asha", "Bharat", "Charan"]
subjects = ["Python", "Maths", "Web"]
marks = [
    [88, 92, 85],
    [76, 81, 79],
    [95, 89, 91]
]

if any(mark < 0 or mark > 100 for row in marks for mark in row):
    print("Invalid marks")
elif not marks or not subjects or any(len(row) != len(subjects) for row in marks):
    print("Invalid marks matrix")
else:
    totals = []
    for name, row in zip(students, marks):
        total = sum(row)
        totals.append(total)
        print(f"{name}: total={total}, average={total / len(row):.2f}")

    top = totals.index(max(totals))
    print("Top student:", students[top])
```

```
for col, subject in enumerate(subjects):
    average = sum(row[col] for row in marks) / len(marks)
    print(f"{subject} average: {average:.2f}")
```

Test Cases

Test Input	Expected Result	Purpose
Asha [88,92,85]; Bharat [76,81,79]; Chaks [95,88,92]; Charan [90,86,75; Charan is top		Student ranking
Any mark = 105	Invalid marks	Range validation
Rows have different lengths	Invalid marks matrix	Matrix structure

Conclusion

The program demonstrates nested lists, loops, sums, averages, and ranking using student and subject data.

8. Smart Canteen Order and Stock Management

Problem Statement

Process canteen orders, calculate order totals, update stock after sales, and report items that need restocking.

Objectives

• Calculate order costs.

• Prevent sales beyond available stock.

• Update stock and identify low-stock items.

Assumptions

• Each product has a unit price and available quantity.

• Order quantities must be non-negative and cannot exceed stock.

• Items with closing stock below 5 units are marked for restocking.

Algorithm

1. Create product records with price and stock.

2. Read or define requested quantities.

3. Validate each requested quantity against stock.

4. Calculate line total and update stock for valid orders.

5. Display bill, closing stock, and restock alerts.

Flowchart

START → Load product stock → Read order quantities → Validate stock → Calculate bill and update stock → Check low stock → Display report → STOP

Pseudocode

```
BEGIN
  total = 0
  FOR each ordered item
    IF quantity < 0 OR quantity > stock: display unavailable
    ELSE
      amount = quantity * price
      total = total + amount
      stock = stock - quantity
      DISPLAY line amount
  DISPLAY total
  FOR each product
    IF stock < 5: DISPLAY restock alert
END
```

Python Source Code

```
products = {
    "idli": {"price": 30, "stock": 20},
    "dosa": {"price": 50, "stock": 10},
    "juice": {"price": 25, "stock": 8}
}
orders = {"idli": 2, "dosa": 1, "juice": 3}
total = 0

for item, quantity in orders.items():
    if item not in products:
        print(item, "is unavailable")
        continue
    product = products[item]
    if quantity < 0 or quantity > product["stock"]:
        print(item, "has insufficient stock or invalid quantity")
        continue
    amount = quantity * product["price"]
    total += amount
    product["stock"] -= quantity
    print(f"{item}: {quantity} x Rs.{product['price']} = Rs.{amount}")
```

```
print(f"Total bill: Rs.{total}")
print("CLOSING STOCK")
for item, product in products.items():
    print(item, ":", product["stock"])
    if product["stock"] < 5:
        print("Restock alert:", item)
```

Test Cases

Test Input	Expected Result	Purpose
2 idli, 1 dosa, 3 juice	Total ■185; closing stocks 18, 9, 5	Normal order
Order 15 dosa when stock is 10	Insufficient stock message	Stock protection
Closing stock 3	Restock alert shown	Low-stock alert

Conclusion

The canteen program links order processing with stock validation, billing, and restocking alerts.

9. Student Academic Risk Classification

Problem Statement

Use student marks and attendance to classify academic risk and report the factors contributing to each classification.

Objectives

• Calculate average marks.

• Check attendance and academic thresholds.

• Classify students and explain risk factors.

Assumptions

• Marks are percentages from 0 to 100.

• Attendance is a percentage from 0 to 100.

• High risk means average below 50 or attendance below 65%; medium risk means average below 65 or attendance below 75%; otherwise low risk.

Algorithm

1. Read each student's marks and attendance.

2. Validate all values.

3. Calculate the average marks.

4. Apply high-, medium-, and low-risk rules in order.

5. Display category and reasons.

Flowchart

START → Read student data → Validate values → Calculate average → Apply risk rules → Display category and reasons → STOP

Pseudocode

```
BEGIN
  FOR each student
    IF invalid marks or attendance: DISPLAY invalid
    ELSE
      average = sum(marks) / count(marks)
      IF average < 50 OR attendance < 65: risk = HIGH
      ELSE IF average < 65 OR attendance < 75: risk = MEDIUM
      ELSE: risk = LOW
      DISPLAY name, average, attendance, risk
END
```

Python Source Code

```
students = [
    ("Asha", [72, 68, 75], 82),
    ("Bharat", [45, 52, 48], 70),
    ("Charan", [38, 44, 41], 60)
]

for name, marks, attendance in students:
    if (not marks or any(m < 0 or m > 100 for m in marks)
            or attendance < 0 or attendance > 100):
        print(name, "- Invalid data")
        continue

    average = sum(marks) / len(marks)
    if average < 50 or attendance < 65:
        risk = "HIGH"
    elif average < 65 or attendance < 75:
        risk = "MEDIUM"
    else:
        risk = "LOW"

    print(f"{name}: average={average:.2f}%, "
          f"attendance={attendance}%, risk={risk}")
```

Test Cases

Test Input	Expected Result	Purpose
Average 72%, attendance 82%	LOW risk	Both measures meet thresholds
Average 48%, attendance 70%	HIGH risk	Average below 50
Average 60%, attendance 80%	MEDIUM risk	Average below 65

Conclusion

The program combines academic performance and attendance into an understandable risk category using explicit thresholds.

10. ATM Menu-Driven Banking Simulation

Problem Statement

Build a menu-driven ATM simulation that supports deposits, withdrawals, balance enquiries, and exit, while validating transactions.

Objectives

• Provide repeated menu options using a loop.

• Update account balance for deposits and withdrawals.

• Reject invalid amounts and withdrawals exceeding the balance.

Assumptions

• The starting balance is ■5,000.

• Transaction amounts must be positive.

• This is an educational simulation and does not connect to a real bank.

Algorithm

1. Set the starting balance.

2. Display the menu repeatedly.

3. For deposit, add a positive amount to the balance.

4. For withdrawal, check the amount and available balance before subtracting.

5. Display balance or exit when requested.

Flowchart

START → Set balance → Display menu → Read choice → Deposit / Withdraw / Check balance / Exit → Validate and update → Repeat or STOP

Pseudocode

```
BEGIN
  balance = 5000
  REPEAT
    DISPLAY menu
    READ choice
    IF choice = 1: READ amount; if amount > 0 add to balance
    ELSE IF choice = 2: READ amount; if 0 < amount <= balance subtract
    ELSE IF choice = 3: DISPLAY balance
    ELSE IF choice = 4: EXIT
    ELSE: DISPLAY invalid choice
  UNTIL choice = 4
END
```

Python Source Code

```
balance = 5000.0

while True:
    print("\nATM MENU")
    print("1. Deposit")
    print("2. Withdraw")
    print("3. Check balance")
    print("4. Exit")
    choice = input("Enter choice: ")

    if choice == "1":
        amount = float(input("Deposit amount: "))
        if amount > 0:
            balance += amount
            print("Deposit successful")
        else:
            print("Amount must be positive")
    elif choice == "2":
        amount = float(input("Withdrawal amount: "))
        if amount <= 0:
```

```
            print("Amount must be positive")
        elif amount > balance:
            print("Insufficient balance")
        else:
            balance -= amount
            print("Withdrawal successful")
    elif choice == "3":
        print(f"Balance: Rs.{balance:.2f}")
    elif choice == "4":
        print("Thank you")
        break
    else:
        print("Invalid choice")
```

Test Cases

Test Input	Expected Result	Purpose
Deposit ■1,000 from ■5,000	Balance becomes ■6,000	Deposit updates balance
Withdraw ■7,000 from ■5,000	Insufficient balance	Over-withdrawal rejected
Choice 4	Program exits	Exit option

Conclusion

The menu-driven simulation demonstrates loops, conditions, user input, and safe validation of basic banking operations.